CS482/495/496 Software Project Proposal: add your tentative project title here

your name(s) here

2025-11-25

1 Client Information

By sharing this client information and the rest of this document, you are stating that this client has provided this project as something they want (not something you created and asked if they wanted), and that they are interested in having you complete this project for your capstone.

- Client name:
- Client title:
- Client email address:
- Client employer:
- How you know the client:

2 Project Description

2.1 Overview

[Add a few paragraphs describing your project succinctly. What problem are you trying to solve, what is the purpose of your project? Why does your client want this project?]

2.2 Key Features

[At this point you should have a basic understanding of your client's needs. List out the key features of the software system the client wants you to build.]

2.3 Why this Project is Interesting

[Why did you decide this project was interesting enough to you to be a capstone project? What about this project is enticing? Why should anyone care?]

2.4 Areas of CS required

[What subfields of computer science seem most likely to be relevant to your project? A capstone must involve multiple.]

2.5 Potential Concerns and Questions

[Is there any aspect of this project that makes you unsure if it will work, either due to your own interests/background, or that you aren't sure if it fits the requirements? Are there questions you have about this project that you want instructor feedback about?]

2.6 Summary of Efforts to Find a Project

(Not necessary for 482) [Briefly list out when/how you've discussed with this client, and if you've discussed with other clients who either didn't work out or didn't respond. If you considered a different project and it didn't work out, why didn't it work out?]

[Most CS495 projects end here. The sections below are for CS482 and CS496 software projects].

2.7 Comparison to Draft

[For CS496 only, focus on highlighting the major differences between the draft proposal in CS495 and this one here. If there are no major differences, you can remove this subsection.]

3 Requirements

3.1 Non-Functional Requirements

[Non-functional requirements are just as important as functional requirements. Dont forget to specify them.]

ID	NFR Title	Category	Description
NFR1	NFR Example 1	Usability	Description of the NFR (it does not follow a user story template)
NFR2	NFR Example 2	Security	Description of the NFR (it does not follow a user story template)

Table 1: Non-Functional requirements

3.2 Functional Requirements (User Stories)

[In CS482, all functional requirements are written as User Stories. In CS496, some projects may use a different template to write the requirements. The table below is an example of writing the Stories. Adapt accordingly to different templates or if you want to display more info.]

ID	Story Title	Points	Description
S1	Story Example 1	5	As a user, I want to write a user story example, so that people will understand them.
S2	Story Example 2	2	As a user, I want to write a user story example, so that people will understand them.

Table 2: Functional requirements as User Stories.

4 System Design

4.1 Architecture

For this project, we will be using the MVC architecture. This architecture will be composed of three layers, each with their own modules.

The Model layer defines data schemas and handles database communication with MongoDB collections via Mongoose. Including UserModel to manage user data such as name, email, and password, EventModel to store calendar event details like title date, time, description. An AuthModel to manage authentication tokens and session validation and a Data Base connection model to connect to the production and test mongo database.

The View layer consists of front-end templates and React components including the Login/Register page, the main Dashboard, EventForm, CaladarView, and ProfilePage

The Controller layer receives user requests, manipulates the Model, and returns a View or data Which could include the UserController, AuthController and EventController

4.2 Diagrams

~~[CS482, on sprints/iterations 2-3, you need to create and update a diagram (check the assignment for which type of diagram). On CS496, since before sprint/iteration 1 you should have a class diagram and keep it up-to-date.]~~

4.3 Technology

For our frontend, we will be using the React framework, and we will be using Vite as our build tool. We will also be using Tailwind Css for our styling. For our backend, we will be using Node.js for our runtime, Express.js for our framework, bcrypt for our password hashing, and JSON Web Token for our authentication. For our database, we will be using MongoDB with Mongoose for schema modeling and database interaction. And for the calendar we will use FullCalendar, an open source JavaScript library for event management and displaying the calendar. We will be using Jest for our testing, along with the build-in mock system inside of Jest. We will be using dotenv to securely load our environment. For our deployment, we will be using Railway. And we are going to implement Github Actions in the main branch to make sure that only viable and approved commits are allowed to be pushed passing all of the tests in the pipeline.

4.4 Coding Standards

We will be using Pascal case for our schemas and file names, and all lowercase for our folder names. For our variable and function names in JS, we will be using Camel case. For our design, we will focus on simplicity, functionality, and ease of use over pure looks.

All code that is pushed to main should have unit tests and coverage that is over 70%. We will all use the project board to communicate which tasks we are working on. When working on the project, we will create a working branch which will only be merged with the main branch when they have been tested and reviewed after each iteration. We will implement a .env file to hide sensitive information. We will follow our user stories and always include the # in our commits so they link to the user story we are working on. We will commit frequently and always include meaningful commit messages. When we find a bug, we will add tests to specifically test that the bug has been eliminated, so we know if that bug ever shows up again. When testing our code, we will use our test database and not the main database. Our code will be object-oriented and modular to improve readability. And if needed we can implement pair programming to tackle big/ complex user stories.

4.5 Data

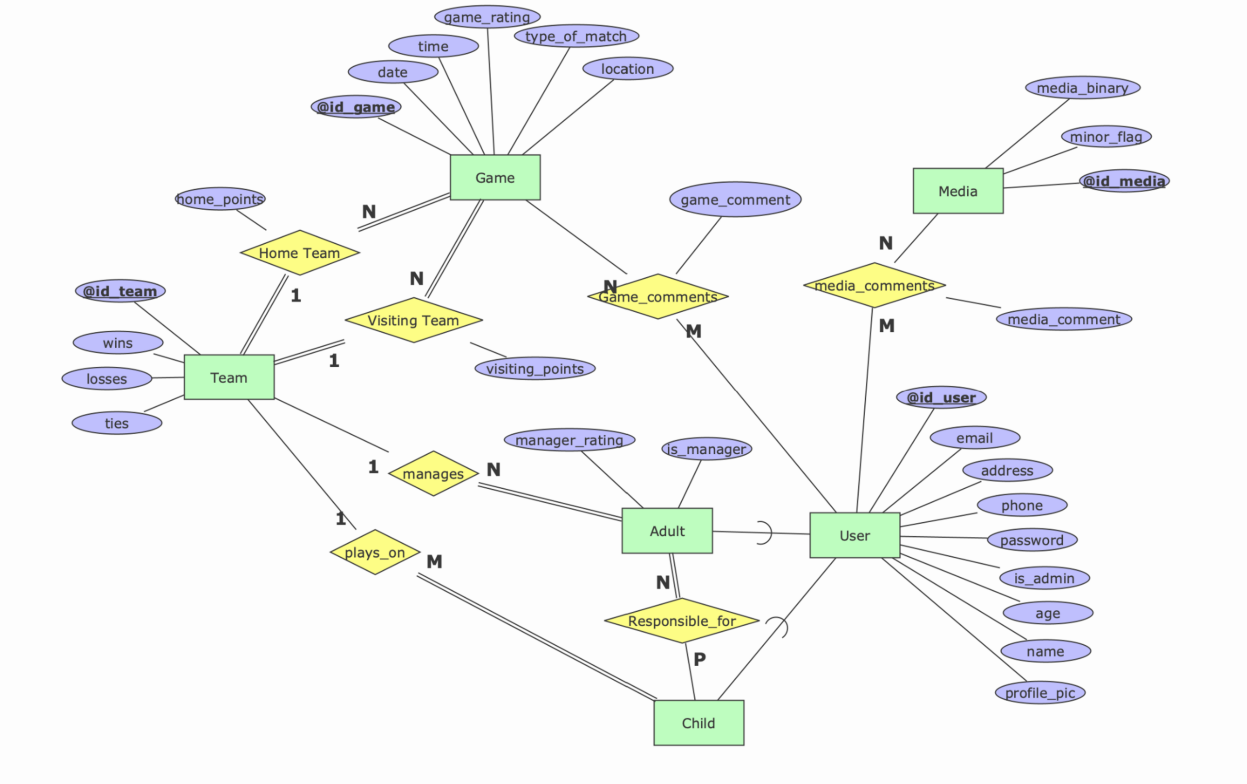


Figure 1: Our ERD

4.6 UI Mocks

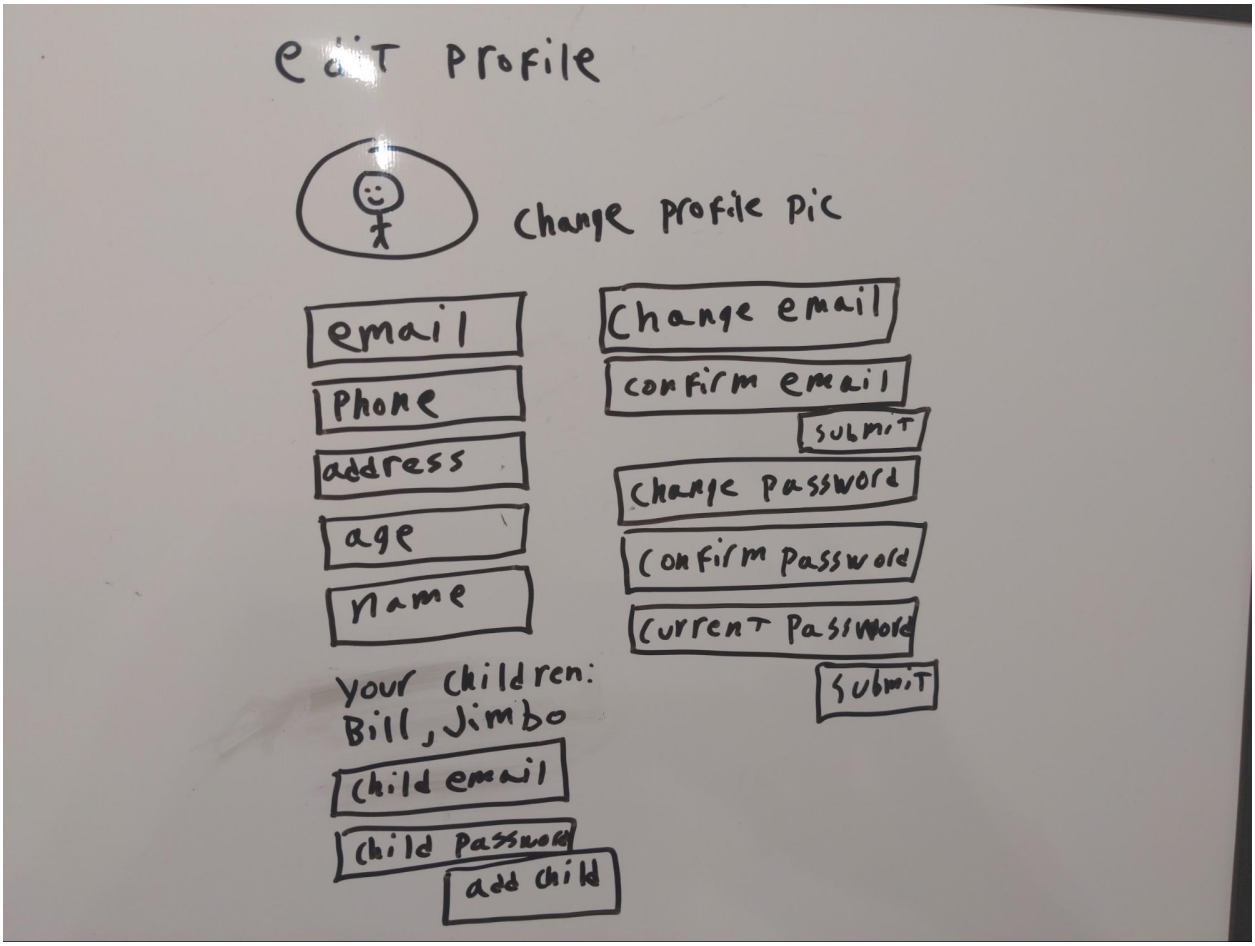


Figure 2: A page to edit your profile

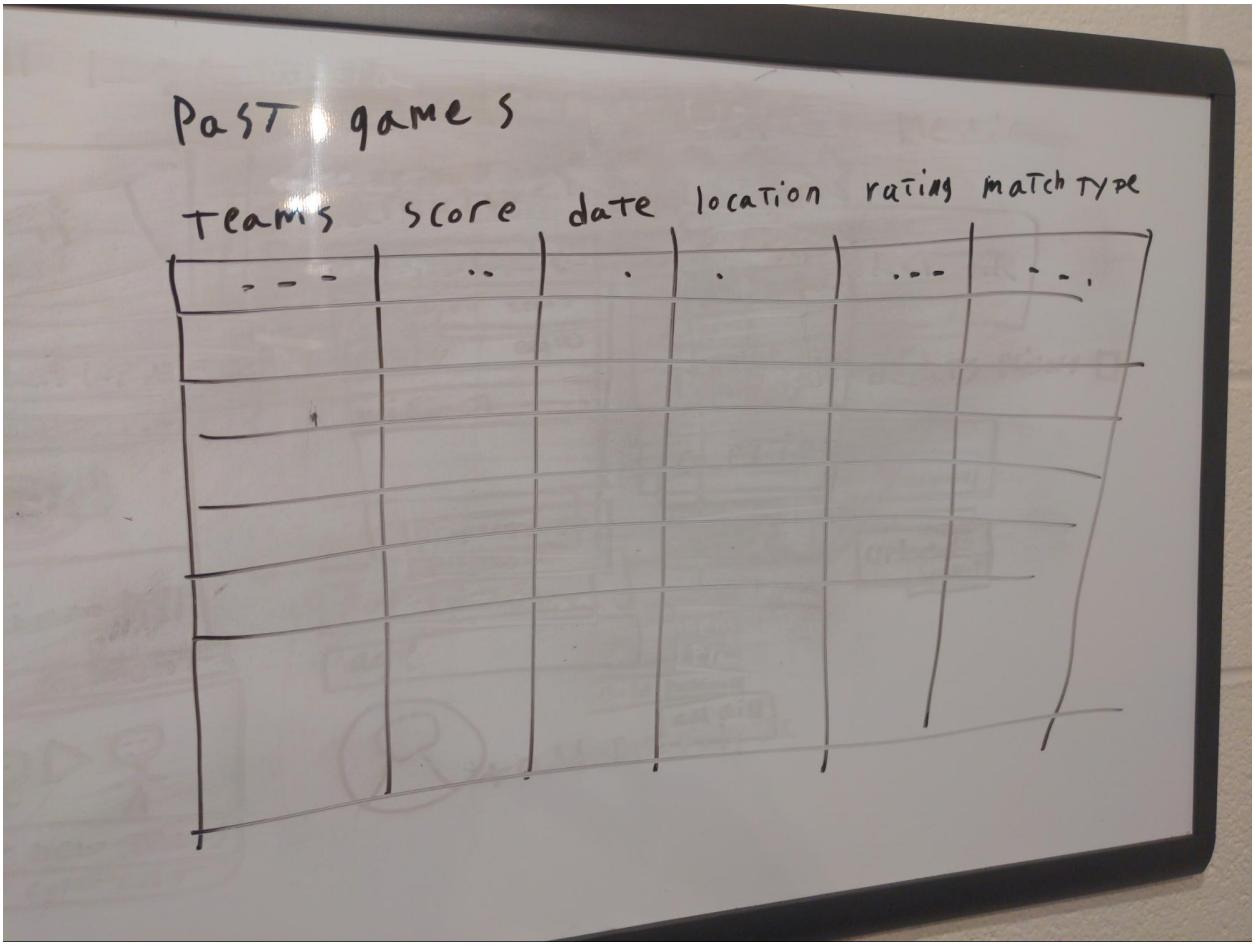


Figure 3: A page to see all past games

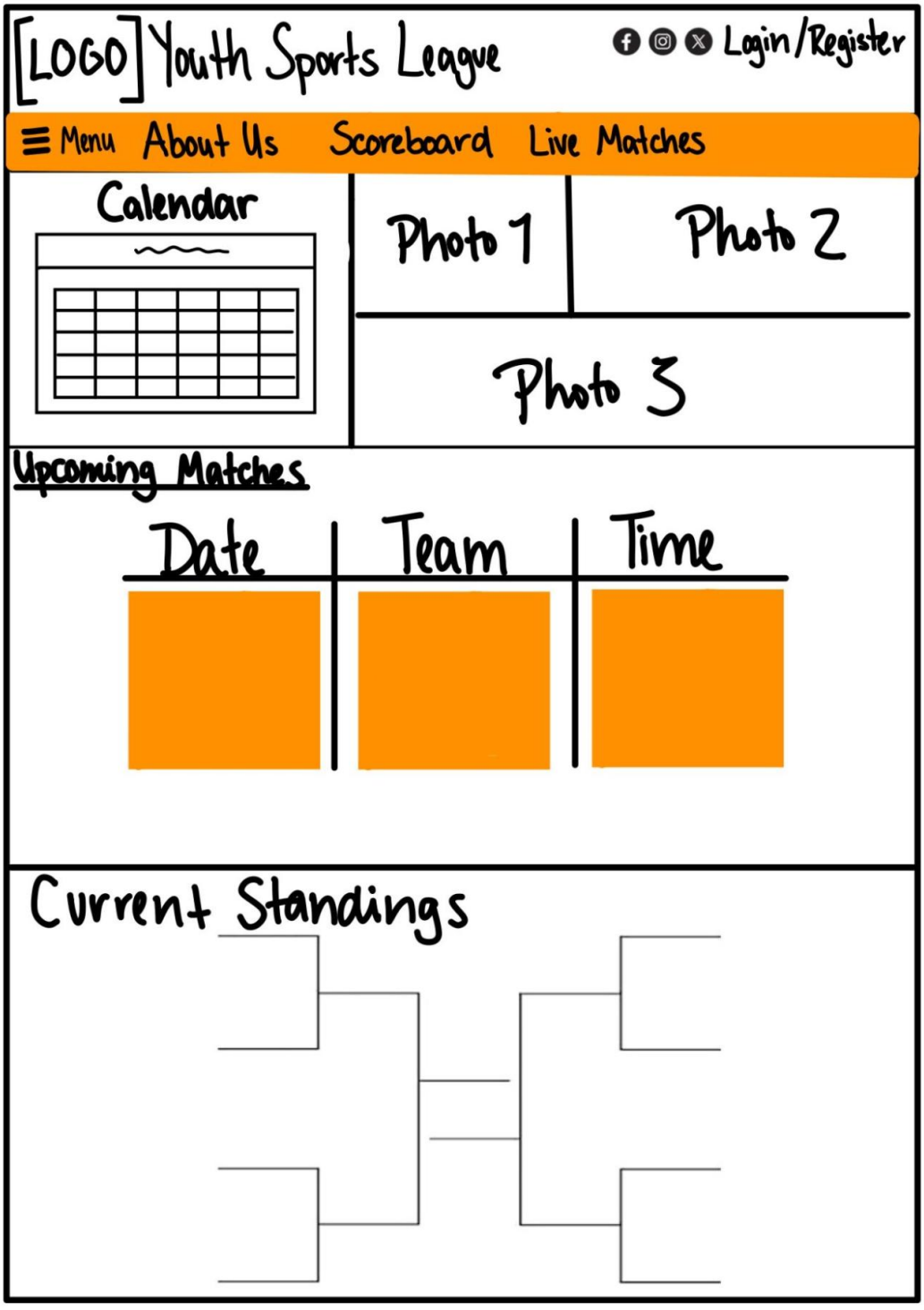


Figure 4: The main landing page with calendar, bracket, user posted images, and upcoming games

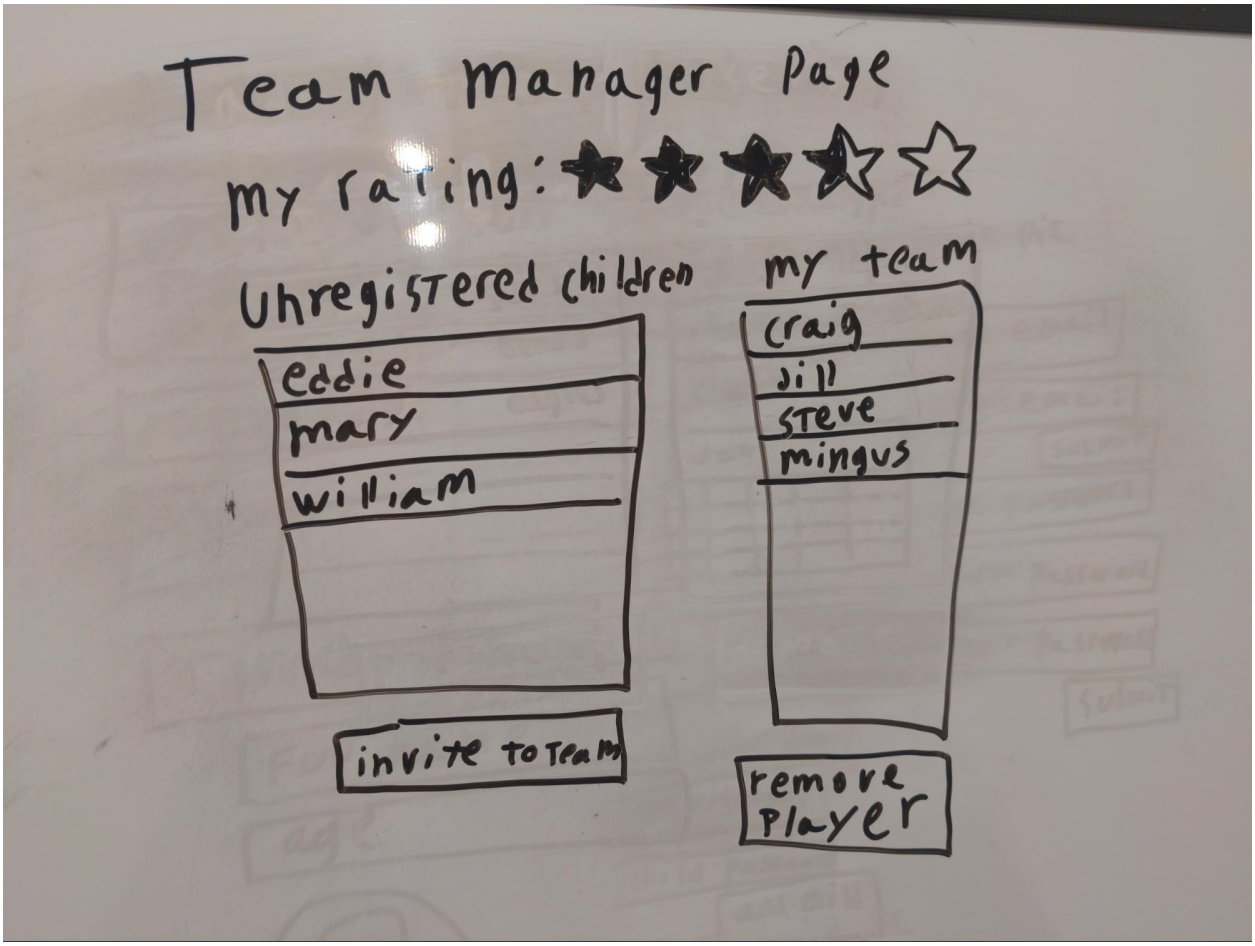


Figure 5: A page for the team manager to see their score and manage their team

Figure 6: An alternative team manager page

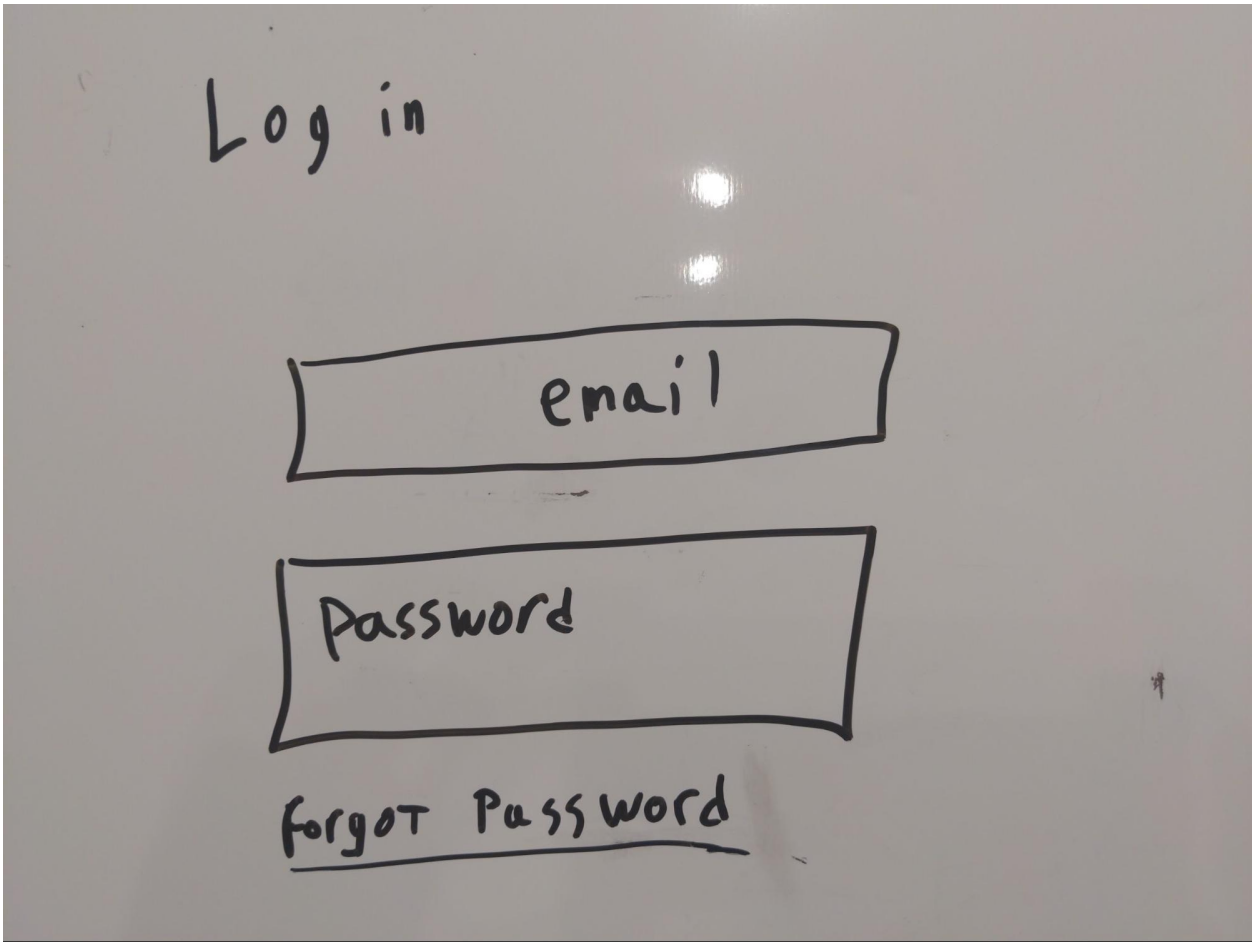


Figure 7: A basic login page

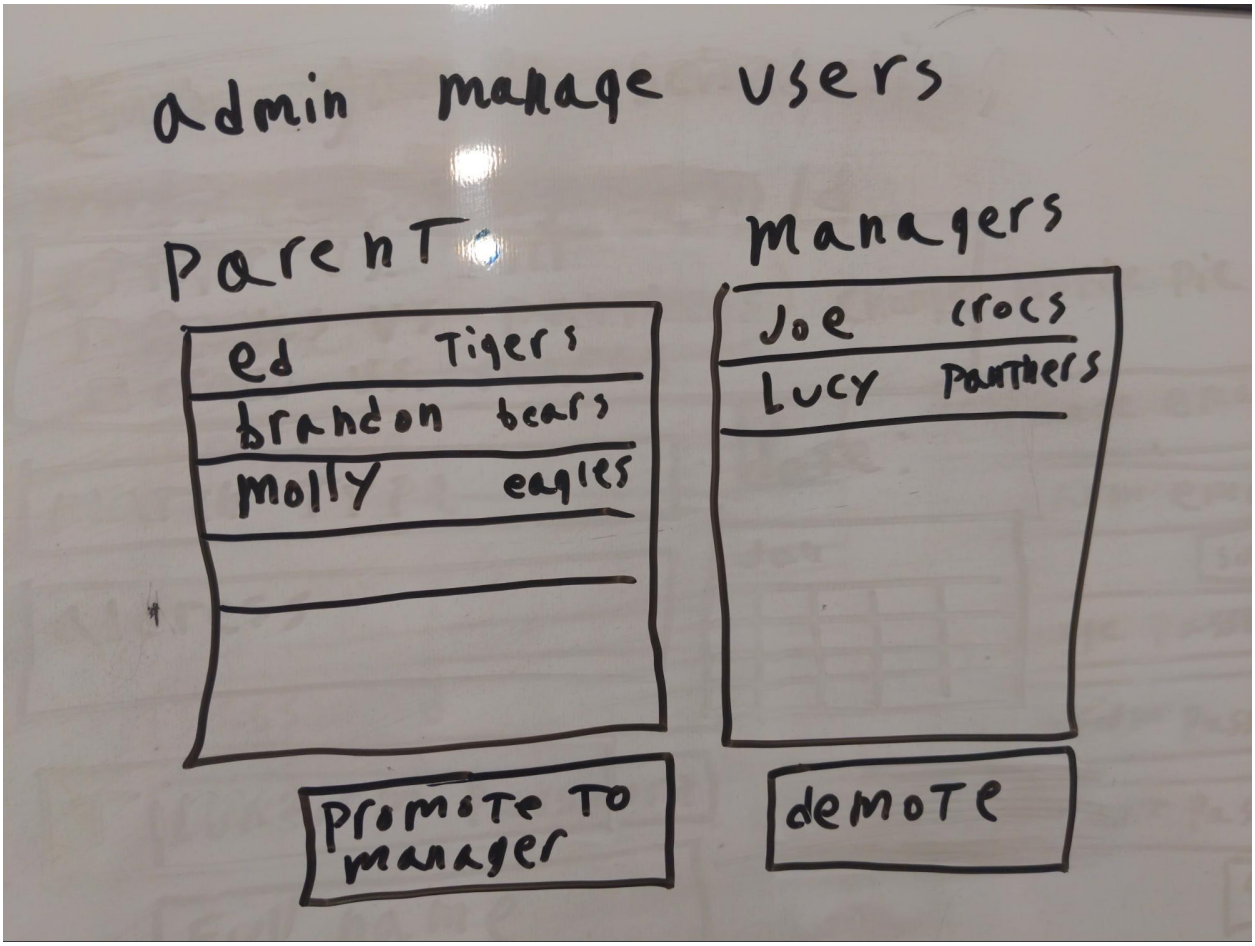


Figure 8: A page for admins to promote and demote team managers

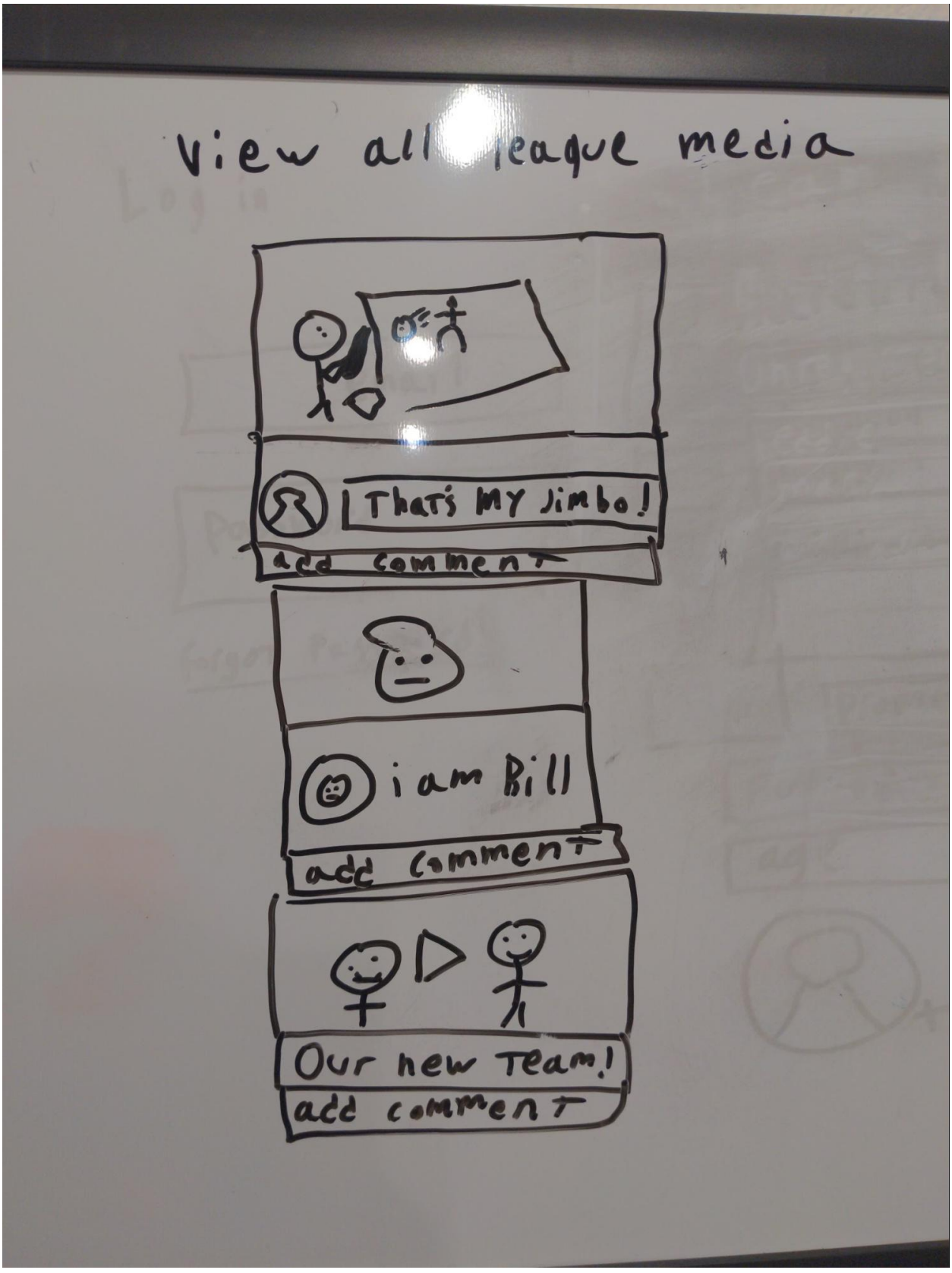


Figure 9: A page to see all previous posts by users and comment on their posts

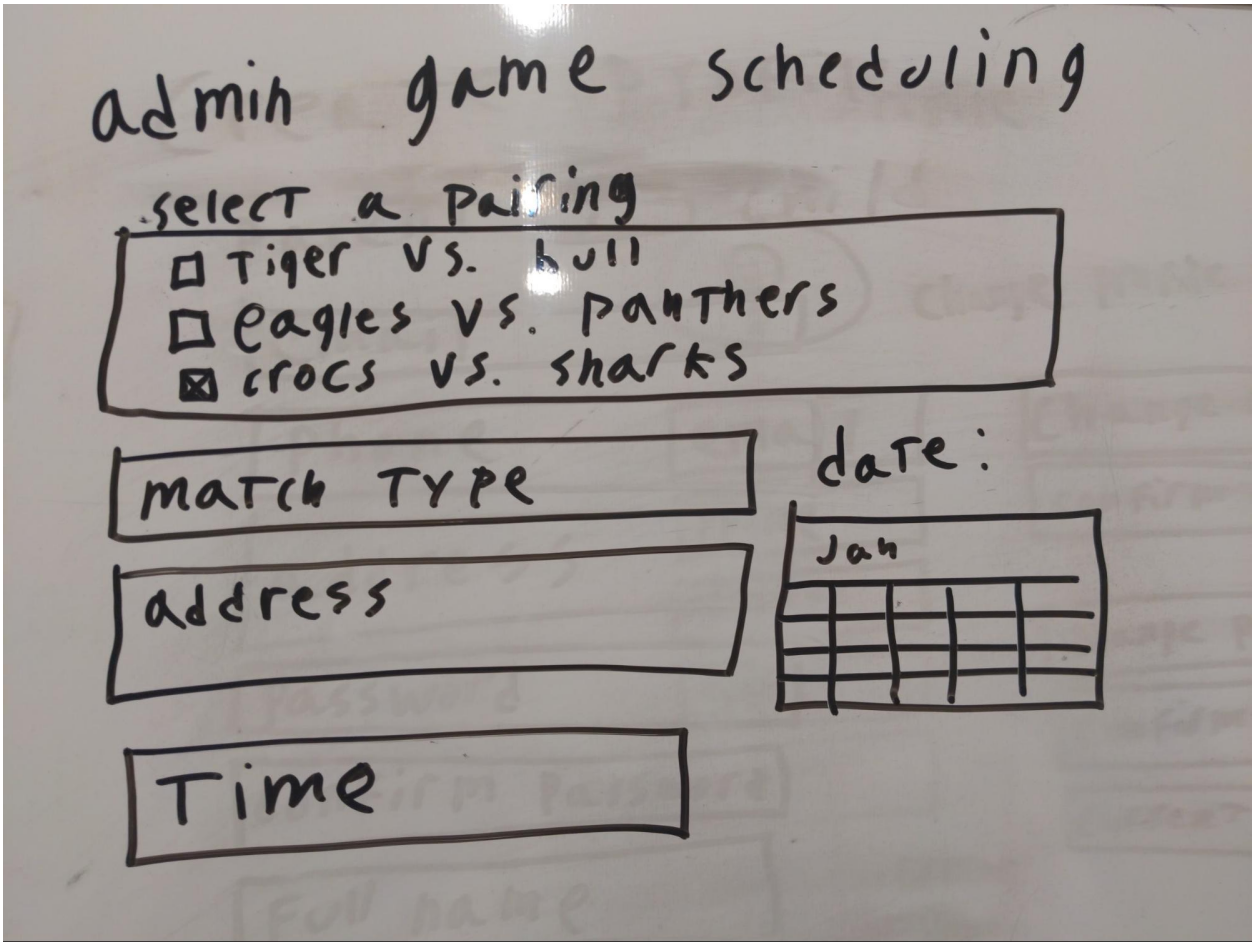


Figure 10: A page for admins to schedule the games that still need to happen for the season

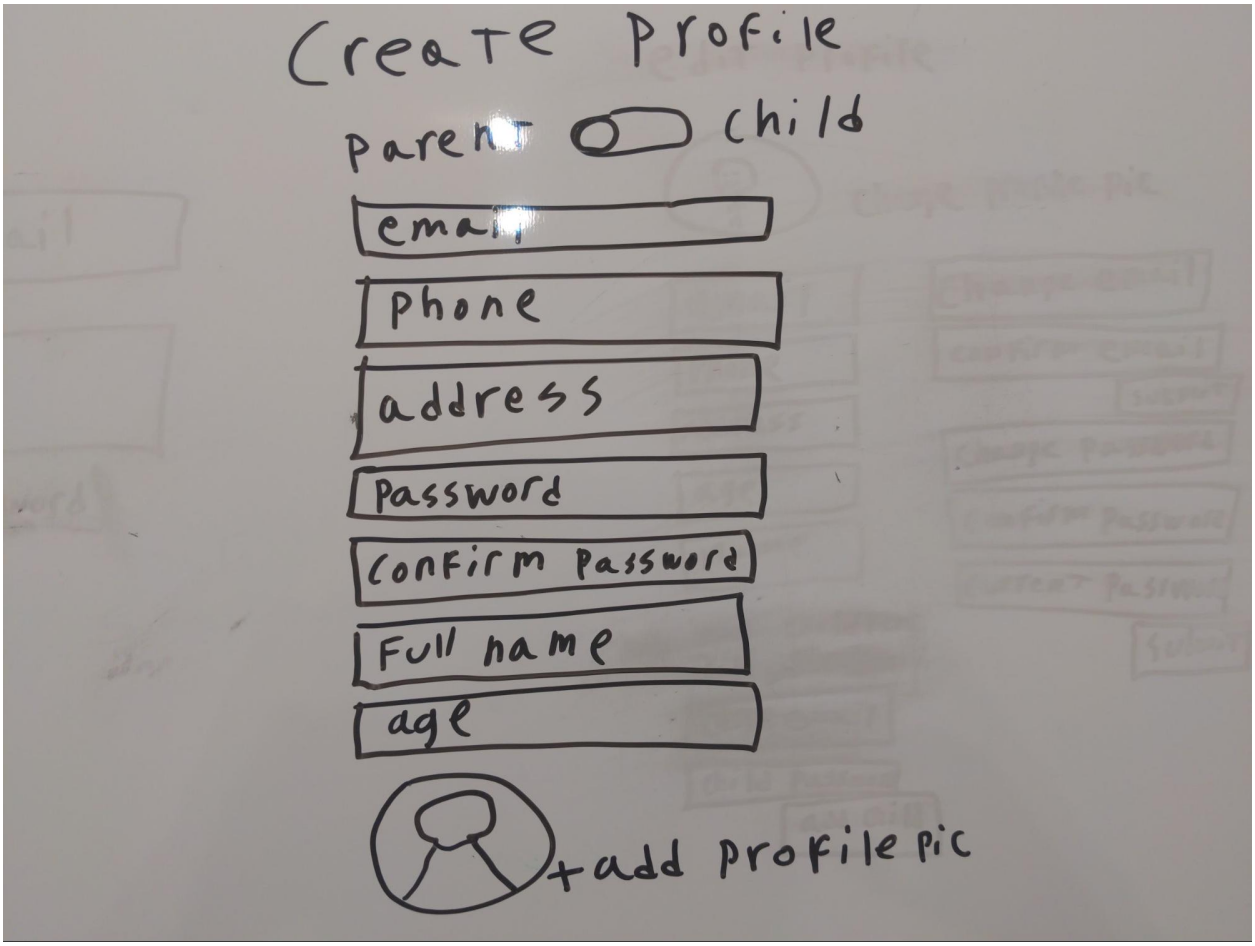


Figure 11: A basic account creation page

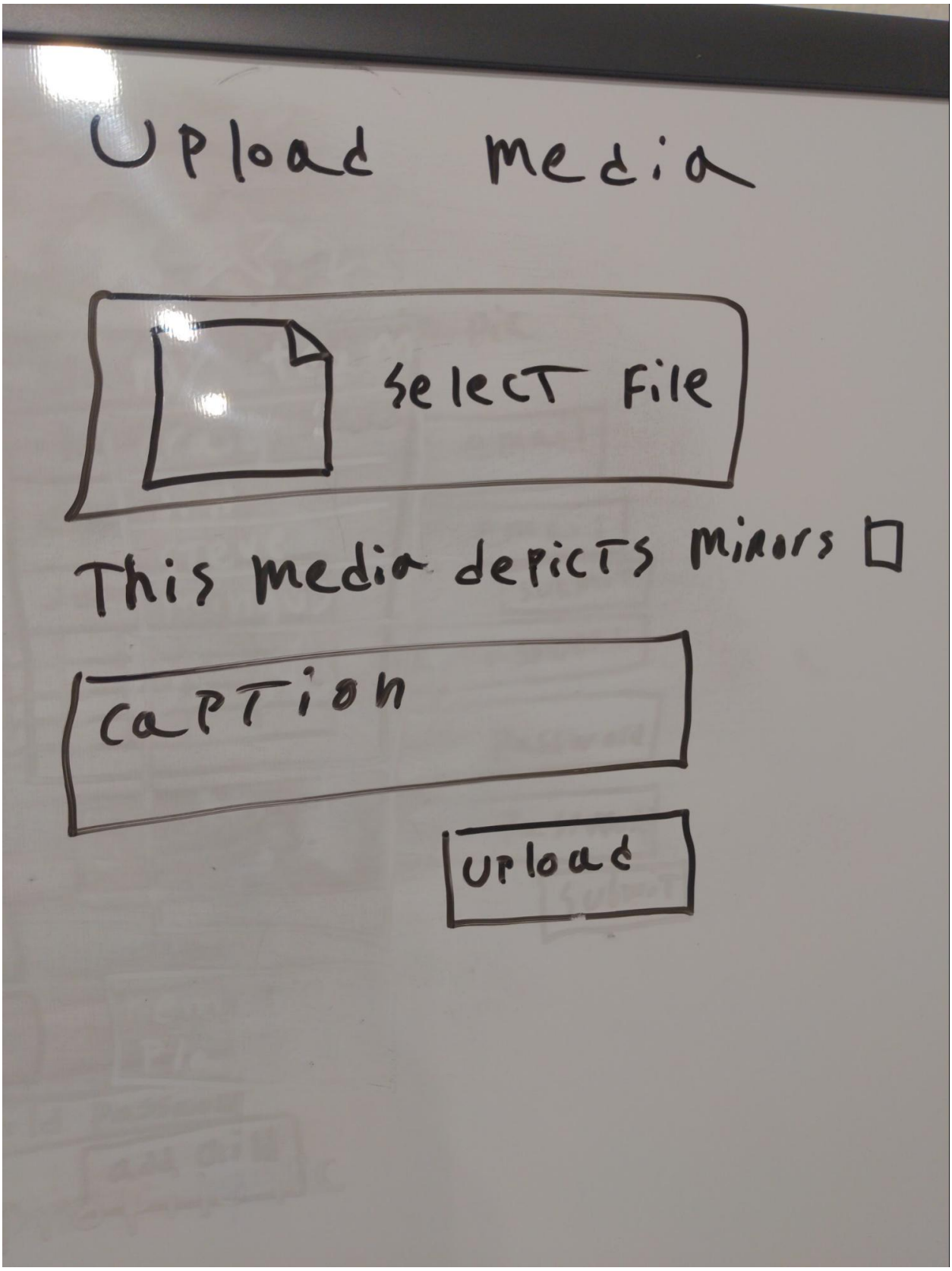


Figure 12: An upload page for uploading pictures and videos of the league

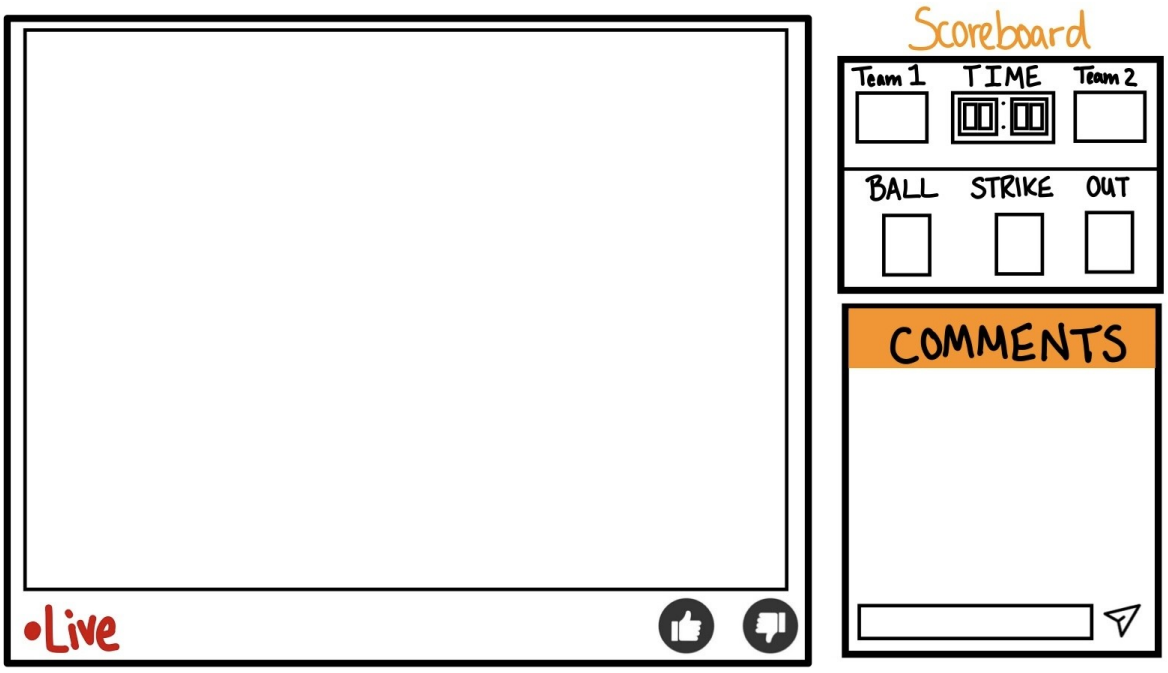


Figure 13: A page to see the current game score, rate the game, and comment your thoughts

5 Iterations

5.1 Iteration Planning

~~[In CS496, you plan all iterations beforehand. In CS482, you update the planning here at each iteration.]~~

Iteration	Dates	Stories	Points
1	01/01 - 02/01	Website Design, E2-Calendar Features, U4-Leaderboard, U3-Scoreboard, A1-SignIn/Out	15
2	02/01 - 03/01	A1-SignIn/Out, P1-Register Parent and Child, A2-Sign up Disclaimer, A3-Promote/Demote, U11-View profile, U4-Leaderboard, U9-Post to landing page, U10-Minor flag, A4 - Enter points	19
3	03/01 - 04/01	U1-View Home Page Info, U7-Seaons, Web-Page UI	5
~~4~~	~~04/01 - 05/01~~	~~S12 Story Title, S13 Story Title, S14 Story Title, S15 Story Title~~	~~19~~
~~5~~	~~05/01 - 06/01~~	~~S16 Story Title, S17 Story Title~~	~~06~~
		Total:	70

Table 3: Iteration Planning for Incremental Deliveries

5.2 Iteration/Sprint 1

5.2.1 Planning

Planned Stories

- Build website and Design (1 pt) – To have a Navbar and Footer setup for each page.
- E2 – Calendar Features (9 pts) – This was our main focus for this iteration, as we wanted to have fully working CRUD operations for a calendar to display events and work with routes and database operations. Now that we have a history of events in the database, we can use the same data and just add scores and ratings for when we want to use it for the live game view.
- U4 – Leaderboard (2 pts) – To show which teams are winning.

Not started?

not sure if started because there are no commits

- U3 - Scoreboard (2 pts)
- A1 - SignIn/Out (1 pts)

Total Points: 15 points

5.2.2 Work Done

In this iteration Jonathan has completed setting up the DB and programming environment the home page, which includes the Navigation bar and footer for every page.

Another story that was completed was a full CRUD calendar. As Crystal worked on the admin page viewing the calendar and being able to click on a date to edit it. Stuart added the EventDao and EventDao Tests. Jonathan worked on the EventController and EventController tests, Server routes/ api calls and Home page Calendar view

A Story being worked on by Mathew is the Leader board as there is a page to view the .json data and css to style the page, but nothing shows up because there are no models supporting this data.

5.2.3 Testing Coverage

```
----------------------|---------|----------|---------|---------|-------------------
File                  | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
----------------------|---------|----------|---------|---------|-------------------
All files             |   97.64 |       70 |     100 |     100 |
 controller           |     100 |      100 |     100 |     100 |
  EventController.js  |     100 |      100 |     100 |     100 |
 model                |   95.34 |       50 |     100 |     100 |
  DbConnection.js     |    90.9 |       50 |     100 |     100 | 6-7
  EventDao.js         |   96.87 |       50 |     100 |     100 | 44
----------------------|---------|----------|---------|---------|-------------------

Test Suites: 2 passed, 2 total
Tests:       21 passed, 21 total
Snapshots:   0 total
Time:        3.544 s
```

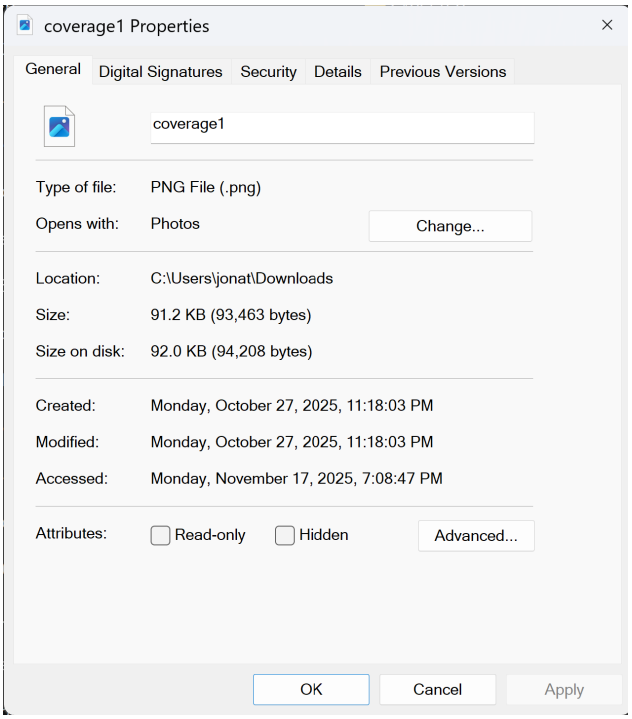


Figure 14: Proof of Iteration 1 Coverage Date - hours after your feedback

I would say that this coverage is good because it goes through every test except for in the Dao when there is no event it just returns null and in the DB connection is says it is not using the testdb even though the testdb works and has been used.

5.2.4 Retroespective & Reflection

Some challenges during this Iteration was communication as we did not really know who was working on what except for a few people. And Time management as some people started early and worked on it little by little while others waited until the last day to start working. And there are few commits in the repo showing what you are working on and making sure that you are on the working branch.

so for next iteration we should commit more often and start earlier so that we are not rushing and have clear communication on what we are doing utilizing the story project board and Discussion tab.

5.3 Iteration/Sprint 2

5.3.1 Planning

Jonathan

- U9 - Post to Landing Page Feed (2 pts)
- U10 - Minor Flag (1 pt)
- A4 - Enter Points (1 pt)

Total: 4 points

Reasoning: I chose these user stories because I had already started learning how to upload an image from the Spike. Using what I have learned, I created the image gallery with image upload functionality. The Minor Flag story ensures that only images of minors are shown when logged in and that an admin can remove inappropriate pictures.

Crystal

- A1 - SignIn/Out (1 pt)
- P1 - Register Parent and Child (2 pts)
- A2 - Sign up Disclaimer (1 pt)

Total: 4 points

Matthew

- A3 - Promote/Demote (1 pt)
- U11 - View Profile (1 pt)
- U4 - Leaderboard (2 pts)

Total: 4 points (or 3 points if Leaderboard is partial)

Stuart

- S3 - Countdown Timer Spike (2 pts)
- U5 - Live Game (5 pts)

Total: 7 points

5.3.2 Work Done

In this iteration, i have completed the Gallery which includes the minor Flag and added editable scoreboards, but once others complete their stories i can go back and officially add the user privileges when viewing the gallery and scoreboards instead of the temporary fix. So, guests cannot see minor flags and cannot edit the scoreboards based on their sessions.

5.3.3 Testing Coverage

```
----------------------|---------|----------|---------|---------|-------------------
File                  | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
----------------------|---------|----------|---------|---------|-------------------
All files             |   98.68 |    84.21 |     100 |     100 |
 controller           |     100 |      100 |     100 |     100 |
  EventController.js  |     100 |      100 |     100 |     100 |
  ImageController.js  |     100 |      100 |     100 |     100 |
 model                |   97.18 |    57.14 |     100 |     100 |
  DbConnection.js     |    90.9 |       50 |     100 |     100 | 6-7
  EventDao.js         |   96.87 |       50 |     100 |     100 | 44
  ImageDao.js         |     100 |      100 |     100 |     100 |
----------------------|---------|----------|---------|---------|-------------------

Test Suites: 4 passed, 4 total
Tests:       43 passed, 43 total
Snapshots:   0 total
Time:        3.283 s
```

this coverage is good enough because it has the same coverage as the previous iteration plus a 100 percent coverage of the newly added ImageDao showing everything is working. Handling tests of inputting images and dealing with the minor flag.

but upon reviewing the files it looks like someone committed code without any testing for the Dao/Controller :(-later on, even for Users.

5.3.4 Retroespective & Reflection

Some of the pitfalls during this iteration would be communication as we did not know what others were planning on doing this iteration and committing their progress to show what they are currently working on.

Another issue was when trying to reword a commit message without the issue link all of the commits after that was somehow duplicated, and it was decided to leave it alone to not further mess up the branch by trying fixing something that had little to no impact. And now added the final git command to implement your new commit message in the discussions tab for future reference, so that this problem does not happen again.

5.4 Iteration/Sprint 3

5.4.1 Planning

Jonathan

- U7 - Seasons (2 pts)
- U1 - View Home Page Info (1 pt)
- Web Pages UI Design (2 pt)

Total: 5 points

Reasoning: I chose these user stories because I had already been working in Figma to create a better looking leader board and I wanted to add more info about the League to the home page instead of just A small calendar and gallery and I have been working on the Teams and wanted to create a Seasons view with A Ui and implementing my already created Scoreboard component, So I was able to reuse that style in the UI for the Seasons.

Crystal

- A2 - Promote/Demote (1 pt)
- A3 - Sign Up Disclaimer (1 pt)
- M1 - Send Invites (2 pts)
- U11 - View Profile (1 pt)

Total: 5 points

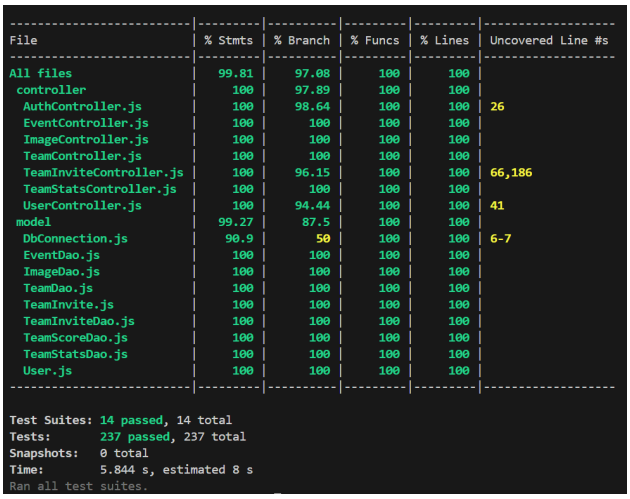
Reasoning: I chose to work on these user stories because I already worked on the Login/SignUp feature and I felt like these stories related well to them. It felt similar to what I had already done and they all connected to the core authentication and user-management flow. I also wanted to do the disclaimer I wasn't able to complete in the previous iteration.

5.4.2 Work Done

Jonathan: I was able to complete the Ui design adding A Leader board UI and seasons Ui and working on component for the home page. And I was able to add info about the Sports league so that someone can better understand what the league is about. And I was able to create a mostly working Seasons backend Creating a page for an admin to create a team that utilizes the users which i would want to use for events and team stats storing all of the scores for each and all games.

Crystal: This iteration I spent a lot of time cleaning up and fixing parts of the login system that I wanted to change from my previous work. Once I sorted that out, I added the sign-up disclaimer so parent users can't complete registration unless they acknowledge it. I moved on to the View Profile feature, making sure users can see their correct information once they're logged in. After that, I added the promote/demote feature for administrators to manage user roles. Finally, I built the send-invites feature so managers can invite players(max 15) to their teams and players can accept or reject invitations. I also added some tests and allowed Team Managers to change the name of their teams.

5.4.3 Testing Coverage



File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	99.81	97.08	100	100	
controller	100	97.89	100	100	
AuthController.js	100	98.64	100	100	26
EventController.js	100	100	100	100	
ImageController.js	100	100	100	100	
TeamController.js	100	100	100	100	
TeamInviteController.js	100	96.15	100	100	66,186
TeamStatsController.js	100	100	100	100	
UserController.js	100	94.44	100	100	41
model	99.27	87.5	100	100	
DbConnection.js	90.9	50	100	100	6-7
EventDao.js	100	100	100	100	
ImageDao.js	100	100	100	100	
TeamDao.js	100	100	100	100	
TeamInvite.js	100	100	100	100	
TeamInviteDao.js	100	100	100	100	
TeamScoreDao.js	100	100	100	100	
TeamStatsDao.js	100	100	100	100	
User.js	100	100	100	100	

Test Suites: 14 passed, 14 total
Tests: 237 passed, 237 total
Snapshots: 0 total
Time: 5.844 s, estimated 8 s
Ran all test suites.

Figure 15: Iteration 3 Coverage

Jonathan: This coverage is good because it has the same coverage as the previous iteration plus a 100 percent coverage of the newly added TeamDao/Controller and TeamStatsDao/Controller showing everything is working. And I was even able to add tests for User.js.
Crystal: I created test coverage for the team invitation system, the authentication and authorization system, user profile management, and the scoring system.

5.4.4 Retroespective & Reflection

Jonathan: Some challenges i found were trying to connect all of the models together as i wanted to create a team with just its name, logo, manager, and players and use the name to use for events to store home team and scores, location, field, inning, and rating and use that for the seasons to store the games and gather all of the scores for team stats. Using all of these models together instead of creating separate models ever time which was why I had to convert the two already created team models into one.

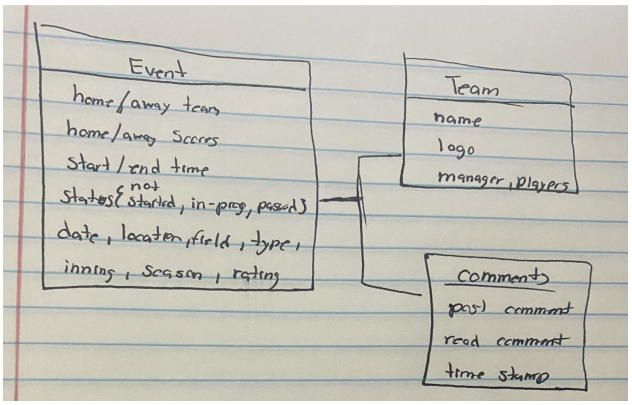
Also need communication and git history...

Crystal: One of the biggest challenges this time was dealing with older login code that didn't work properly. It ended up being more of a connection to the server issue than a code problem. I also had to do a lot of logging in and out of different roles to see if everything was working properly and doing a bunch of changes once a role didn't do what it was supposed to do. A big takeaway for me is how important communication and clean Git history are. I definitely need to make more frequent commits and tie them clearly to stories to make the workflow smoother.

5.5 Modeling

that has the main classes responsible for the Live Scoring and Live Matches

current state of Live matches is unknown but for this For an event we would have the strart time and then the length of a game to get the end time and in that window the game will be in a state of in-progress where the admin can change the scoreboard points and when the time is up the game will be in a state of passed and stores the event data on the game. And for the comments have the input and with time stamp so you know which messages appeared when and can be replayed at the same time intervals based on the time stamps and for the game while in the in progress state a user can Like/Dislike the game.



5.6 Iteration/Sprint 4

[CS496 has 5 sprints. CS482 only has only 3 sprints (remove Iterations 4 and 5 from this doc if you are writing a doc for 482]

5.6.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.6.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.6.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.6.4 Retroespective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

5.7 Iteration/Sprint 5

5.7.1 Planning

[Which stories did you plan for this iteration/sprint. Add the total points for this plan. You can also explain the reason behind your planning, and what major feature(s) your team is focusing on delivering by

completing these stories. You may use a table for a summary display of the planning, but elaborate in text more detail in your focus and feature plan.]

5.7.2 Work Done

[Which stories did you complete in this iteration/sprint. Which ones did you partially complete? Who worked on which story? You may elaborate in paragraph(s) to add more detail about the work done.]

5.7.3 Testing Coverage

[Testing is very important. Show your coverage here. Is this coverage good enough? Explain why you think so. Is it not good enough? Explain a plan to increase the coverage. You may also elaborate on why some artifacts do not undergo much testing. If the testing changed from the last iteration, explain the reasons.]

5.7.4 Retroespective & Reflection

[What were the pitfalls, challenges, and issues you had in this iteration? How can you address them to improve the process in the next iteration? Did anything not go according to plan? Why so and how to avoid the same mistake? Write a personal reflection on what you learned in this iteration (even if a small technical thing like Database storage).]

6 Final Remarks

6.1 Overall Progress

[Have you completed everything? If so, present evidence on how you brought value to your client, and the overall client satisfaction. Otherwise, estimate how much progress you done and how long it would take to finish this project.]

6.2 Project Reflection

[Your personal reflection on the project. What lessons did you learned. What would you have done differently. How can you do better work in future projects? You may write this as a team or per person (or both)]

Appendix

[Appendix section if needed]